

14.4 La più lunga sottosequenza comune (LCS)

In biologia a volte serve confrontare il DNA di due (o più) organismi diversi. Una sequenza di DNA è costituita da una sequenza di molecole chiamate **basi**; le basi possibili sono l'adenina, la citosina, la guanina e la timina. Rappresentando ciascuna di queste basi con la sua iniziale, un filamento di DNA può essere rappresentato con una stringa sull'insieme finito di 4 elementi $\{A, C, G, T\}$ (la definizione di stringa si trova nell'Appendice C). Per esempio, il DNA di un organismo potrebbe essere $S_1 = \text{ACCGGTCGAGTGC GCGGAAGCCGGCCGAA}$, mentre il DNA di un altro organismo potrebbe essere $S_2 = \text{GTCGTTCCGAATGCCGTTGCTCTGTAAA}$. Un motivo per confrontare due filamenti di DNA potrebbe essere per determinare quanto "simili" siano i due filamenti, per misurare il grado di parentela tra i due organismi. La somiglianza può ed è definita in vari modi. Per esempio, potremmo dire che due filamenti di DNA si somigliano se uno è una sottostringa dell'altro (il Capitolo 32 analizza alcuni algoritmi che risolvono questo problema). Nel nostro esempio, S_1 non è una sottostringa di S_2 né S_2 è una sottostringa di S_1 . In alternativa, potremmo dire che due filamenti di DNA si somigliano se il numero di modifiche richieste per trasformare uno nell'altro è piccolo (il Problema 14-5 analizza questa idea). Un altro modo ancora per misurare la somiglianza tra i filamenti S_1 e S_2 consiste nel trovare un terzo filamento S_3 le cui basi si presentano in ciascu-

¹ Nel *Romeo e Giulietta* di Shakespeare, Giulietta Capuleti prende una decisione affrettata e decisamente subottimale. [N.d.T.]

na delle stringhe S_1 e S_2 . Queste basi devono presentarsi nello stesso ordine, ma senza essere necessariamente consecutive. Maggiore è la lunghezza del filamento S_3 che riusciamo a trovare, tanto più S_1 e S_2 si somigliano. Nel nostro esempio, il filamento S_3 più lungo possibile è GTCGTCGGAAGCCGGCCGAA.

Formalizziamo quest'ultima definizione di somiglianza tramite il problema della più lunga sottosequenza comune. Una sottosequenza di una data sequenza è la sequenza stessa alla quale sono stati tolti zero o più elementi. Formalmente, data una sequenza $X = \langle x_1, x_2, \dots, x_m \rangle$, un'altra sequenza $Z = \langle z_1, z_2, \dots, z_k \rangle$ è una **sottosequenza** di X se esiste una successione strettamente crescente $\langle i_1, i_2, \dots, i_k \rangle$ di indici di X tale che per ogni $j = 1, 2, \dots, k$, si ha $x_{i_j} = z_j$. Per esempio, $Z = \langle B, C, D, B \rangle$ è una sottosequenza di $X = \langle A, B, C, B, D, A, B \rangle$ e la successione corrispondente di indici è $\langle 2, 3, 5, 7 \rangle$.

Date due sequenze X e Y , diciamo che una sequenza Z è una **sottosequenza comune** di X e Y se Z è una sottosequenza di entrambe le sequenze X e Y . Per esempio, se $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$, allora la sequenza $\langle B, C, A \rangle$ è una sottosequenza comune di X e Y . Tuttavia, la sequenza $\langle B, C, A \rangle$ non è una **più lunga sottosequenza comune** (*longest common subsequence* o **LCS**) di X e Y , perché ha lunghezza 3 e la sequenza $\langle B, C, B, A \rangle$, anch'essa comune a X e Y , ha lunghezza 4. La sequenza $\langle B, C, B, A \rangle$ è una LCS di X e Y , come pure la sequenza $\langle B, D, A, B \rangle$, perché non esiste una sottosequenza comune di lunghezza 5 o più.

Nel **problema della più lunga sottosequenza comune** sono date due sequenze $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ e si vuole trovare una sottosequenza di lunghezza massima che è comune a X e Y . Questo paragrafo mostra come risolvere efficientemente il problema della ricerca della più lunga sottosequenza comune applicando la programmazione dinamica.

Fase 1: caratterizzare la più lunga sottosequenza comune

Possiamo risolvere il problema della più lunga sottosequenza comune in modo esaustivo: enumeriamo tutte le sottosequenze di X e controlliamole singolarmente sottosequenze per vedere se sono anche sottosequenze di Y , tenendo traccia della più lunga trovata. Ogni sottosequenza di X corrisponde a un sottoinsieme degli indici $\{1, 2, \dots, m\}$ di X . Poiché X ha 2^m sottosequenze, questo approccio richiede un tempo esponenziale, il che lo rende inutilizzabile per sequenze lunghe.

Tuttavia, il problema della più lunga sottosequenza comune gode della proprietà della sottostruttura ottima, come dimostra il seguente teorema. Infatti, le classi naturali di sottoproblemi corrispondono a coppie di "prefissi" delle due sequenze di input. Più precisamente, data una sequenza $X = \langle x_1, x_2, \dots, x_m \rangle$, diciamo che, per $i = 0, 1, \dots, m$, la sottosequenza $X_i = \langle x_1, x_2, \dots, x_i \rangle$ è l' i -esimo **prefisso** di X . Per esempio, se $X = \langle A, B, C, B, D, A, B \rangle$, allora $X_4 = \langle A, B, C, B \rangle$ mentre X_0 è la sequenza vuota.

Teorema 14.1 (Sottostruttura ottima di una LCS)

Siano $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ due sequenze, e sia $Z = \langle z_1, z_2, \dots, z_k \rangle$ una qualsiasi più lunga sottosequenza comune di X e Y .

1. Se $x_m = y_n$, allora $z_k = x_m = y_n$ e Z_{k-1} è una più lunga sottosequenza comune di X_{m-1} e Y_{n-1} .
2. Se $x_m \neq y_n$ e $z_k \neq x_m$, allora Z è una più lunga sottosequenza comune di X_{m-1} e Y .

3. Se $x_m \neq y_n$ e $z_k \neq y_n$, allora Z è una più lunga sottosequenza comune di X e Y_{n-1} .

Dimostrazione 1. Se $z_k \neq x_m$, allora potremmo concatenare $x_m = y_n$ a Z per ottenere una sottosequenza comune di X e Y di lunghezza $k + 1$, contraddicendo l'ipotesi che Z sia una **più lunga** sottosequenza comune di X e Y . Quindi, deve essere $z_k = x_m = y_n$. Ora, il prefisso Z_{k-1} è una sottosequenza comune di X_{m-1} e Y_{n-1} di lunghezza $k - 1$. Vogliamo dimostrare che questo prefisso è una più lunga sottosequenza comune. Supponiamo per assurdo che ci sia una sottosequenza comune W di X_{m-1} e Y_{n-1} di lunghezza maggiore di $k - 1$. Allora, concatenando $x_m = y_n$ a W si ottiene una sottosequenza comune di X e Y la cui lunghezza è maggiore di k , che è una contraddizione.

2. Se $z_k \neq x_m$, allora Z è una sottosequenza comune di X_{m-1} e Y . Se esistesse una sottosequenza comune W di X_{m-1} e Y di lunghezza maggiore di k , allora W sarebbe anche una sottosequenza comune di X_m e Y , contraddicendo l'ipotesi che Z sia una più lunga sottosequenza comune di X e Y .

3. La dimostrazione è simmetrica a quella del punto 2.

La caratterizzazione delle più lunghe sottosequenze comuni del Teorema 14.1 afferma che una più lunga sottosequenza comune di due sequenze contiene al suo interno una più lunga sottosequenza comune di certi prefissi delle due sequenze. Quindi, il problema della più lunga sottosequenza comune gode della proprietà della sottostruttura ottima. Vedremo tra un momento che una soluzione ricorsiva gode anche della proprietà dei sottoproblemi sovrapposti.

Fase 2: una soluzione ricorsiva

Il Teorema 14.1 implica che ci sono uno o due sottoproblemi da esaminare per trovare una più lunga sottosequenza comune di $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$. Se $x_m = y_n$, dobbiamo trovare una più lunga sottosequenza comune di X_{m-1} e Y_{n-1} . Se concateniamo $x_m = y_n$ a questa più lunga sottosequenza comune, allora otteniamo una più lunga sottosequenza comune di X e Y . Se $x_m \neq y_n$, allora dobbiamo risolvere due sottoproblemi: trovare una più lunga sottosequenza comune di X_{m-1} e Y , e di X e Y_{n-1} . La più lunga di queste due sottosequenze è una più lunga sottosequenza comune di X e Y . Poiché questi casi esauriscono tutte le possibilità, sappiamo che all'interno di una più lunga sottosequenza comune di X e Y deve essere utilizzata una delle soluzioni ottime dei sottoproblemi.

Il problema della più lunga sottosequenza comune presenta sottoproblemi sovrapposti. Ecco perché. Per trovare una più lunga sottosequenza comune di X e Y , potrebbe essere necessario trovare sia una più lunga sottosequenza comune di X e Y_{n-1} che una di X_{m-1} e Y . Tuttavia, ciascuno di questi sottoproblemi ha come sottosottoproblema quello di trovare una più lunga sottosequenza comune di X_{m-1} e Y_{n-1} . Molti altri sottoproblemi condividono sottosottoproblemi.

Come nel problema della moltiplicazione di una catena di matrici, risolvere il problema della più lunga sottosequenza comune in modo ricorsivo richiede la definizione di una ricorrenza per il valore di una soluzione ottima. Definiamo $c[i, j]$ come la lunghezza di una più lunga sottosequenza comune delle sequenze X_i e Y_j . Se $i = 0$ o $j = 0$, una delle sequenze ha lunghezza 0, e quindi la più lunga sottosequenza comune ha lunghezza 0. La sottostruttura ot-

tima del problema della più lunga sottosequenza comune consente di scrivere la formula ricorsiva

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0, \\ c[i-1, j-1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j, \\ \max\{c[i, j-1], c[i-1, j]\} & \text{se } i, j > 0 \text{ e } x_i \neq y_j. \end{cases} \quad (14.9)$$

In questa formulazione ricorsiva c'è una condizione del problema che riduce il numero di sottoproblemi da considerare. Quando $x_i = y_j$, possiamo e dobbiamo considerare soltanto il sottoproblema di trovare la più lunga sottosequenza comune di X_{i-1} e Y_{j-1} . Altrimenti, dobbiamo considerare soltanto i due sottoproblemi relativi alla ricerca di una più lunga sottosequenza comune di X_i e Y_{j-1} e di X_{i-1} e Y_j . Negli algoritmi basati sulla programmazione dinamica che abbiamo già esaminato, ovvero quello per il taglio delle aste e per la moltiplicazione di una catena di matrici, nessun sottoproblema è stato escluso a causa delle condizioni del problema. Trovare una più lunga sottosequenza comune non è l'unico algoritmo basato sulla programmazione dinamica che esclude dei sottoproblemi in base a delle condizioni del problema. Per esempio, anche il problema della distanza di editing (si veda il Problema 14-5) ha questa caratteristica.

Fase 3: calcolare la lunghezza di una LCS

Utilizzando l'equazione (14.9) potremmo scrivere facilmente un algoritmo ricorsivo per calcolare la lunghezza di una più lunga sottosequenza comune di due sequenze, che però richiede un tempo esponenziale. Tuttavia, poiché il problema ha solo $\Theta(mn)$ sottoproblemi differenti (ovvero calcolare $c[i, j]$ per $0 \leq i \leq m$ e $0 \leq j \leq n$), possiamo utilizzare la programmazione dinamica per calcolare le soluzioni con un metodo bottom-up. La procedura LCS-LENGTH riceve come input due sequenze $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$ e le loro lunghezze.

LCS-LENGTH(X, Y, m, n)

```

1  siano  $b[1 : m, 1 : n]$  e  $c[0 : m, 0 : n]$  due nuove tabelle
2  for  $i = 1$  to  $m$ 
3       $c[i, 0] = 0$ 
4  for  $j = 0$  to  $n$ 
5       $c[0, j] = 0$ 
6  for  $i = 1$  to  $m$ 
7      for  $j = 1$  to  $n$  // Calcola i valori della tabella per righe
8          if  $x_i == y_j$ 
9               $c[i, j] = c[i-1, j-1] + 1$ 
10              $b[i, j] = "\nwarrow"$ 
11         elseif  $c[i-1, j] \geq c[i, j-1]$ 
12              $c[i, j] = c[i-1, j]$ 
13              $b[i, j] = "\uparrow"$ 
14         else  $c[i, j] = c[i, j-1]$ 
15              $b[i, j] = "\leftarrow"$ 
16  return  $c$  e  $b$ 
```

Memorizza i valori $c[i, j]$ in una tabella $c[0 : m, 0 : n]$, e ne calcola gli elementi come nella **rappresentazione per righe**, ovvero la procedura inserisce i valori

nella prima riga di c da sinistra a destra, poi nella seconda riga e così via. La procedura utilizza anche la tabella $b[1 : m, 1 : n]$ per semplificare la costruzione di una soluzione ottima. Intuitivamente, $b[i, j]$ punta al valore della tabella che corrisponde alla soluzione ottima del sottoproblema che è stato scelto per calcolare $c[i, j]$. La procedura restituisce le tabelle b e c , e $c[m, n]$ contiene la lunghezza di una più lunga sottosequenza di X e Y . La Figura 14.8 mostra le tabelle prodotte da LCS-LENGTH con le sequenze $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$. Il tempo di esecuzione della procedura è $\Theta(mn)$, perché il calcolo di ogni elemento della tabella richiede un tempo $\Theta(1)$.

Fase 4: costruire una LCS

Avendo a disposizione la tabella b restituita dalla procedura LCS-LENGTH, possiamo costruire rapidamente una più lunga sottosequenza comune delle sequenze $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$. Iniziamo da $b[m, n]$ e attraversiamo la tabella seguendo le frecce. Ogni volta che incontriamo la freccia " $\nwarrow$ " in $b[i, j]$, allora $x_i = y_j$ è un elemento della più lunga sottosequenza comune trovata da LCS-LENGTH. In questo modo incontriamo gli elementi della più lunga sottosequenza comune in ordine inverso. La procedura ricorsiva PRINT-LCS stampa una più lunga sottosequenza comune di X e Y nell'ordine corretto. La chiamata iniziale è PRINT-LCS(b, X, m, n). Per la tabella b mostrata nella Figura 14.8, questa procedura stampa **BCBA**. La procedura richiede un tempo $O(m+n)$, perché a ogni chiamata ricorsiva decrementa almeno uno dei contatori i o j .

PRINT-LCS(b, X, i, j)

```

1  if  $i == 0$  or  $j == 0$ 
2      return // La LCS ha lunghezza 0
3  if  $b[i, j] == "\nwarrow"$ 
4      PRINT-LCS( $b, X, i-1, j-1$ )
5      stampa  $x_i$  // Uguale a  $y_j$ 
6  elseif  $b[i, j] == "\uparrow"$ 
7      PRINT-LCS( $b, X, i, j-1$ )
8  else PRINT-LCS( $b, X, i, j-1$ )
```

Migliorare il codice

Dopo aver sviluppato un algoritmo, spesso ci accorgiamo che è possibile migliorarlo in relazione al tempo di esecuzione o alla quantità di memoria utilizzata. Alcune modifiche possono semplificare il codice e migliorare i fattori costanti, ma non producono miglioramenti asintotici delle prestazioni. Altre modifiche possono portare a sostanziali risparmi asintotici di tempo e memoria.

Per esempio, nell'algoritmo della più lunga sottosequenza comune possiamo eliminare completamente la tabella b . Ogni $c[i, j]$ dipende soltanto da altri tre valori nella tabella c : $c[i-1, j-1]$, $c[i-1, j]$ e $c[i, j-1]$. Dato il valore di $c[i, j]$, possiamo determinare in tempo $O(1)$ quale di questi tre valori è stato utilizzato per calcolare $c[i, j]$, senza ispezionare la tabella b . Quindi, possiamo ricostruire una più lunga sottosequenza comune in tempo $O(m+n)$ utilizzando una procedura simile a PRINT-LCS (l'Esercizio 14.4-2 chiede di scrivere lo pseudocodice). Sebbene questo metodo permetta di risparmiare uno spazio $\Theta(mn)$ in memoria, lo spazio ausi-

	j	0	1	2	3	4	5	6
	y_j	B	D	C	A	B	A	
0	x_i	0	0	0	0	0	0	0
1	A	0	0	0	0	1	1	1
2	B	0	1	1	1	1	2	2
3	C	0	1	1	2	2	2	2
4	B	0	1	1	2	2	3	3
5	D	0	1	2	2	2	3	3
6	A	0	1	2	2	3	3	4
7	B	0	1	2	2	3	4	4

Figura 14.8 Le tabelle c e b calcolate da LCS-LENGTH con le sequenze $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$. La casella nella riga i e colonna j contiene il valore di $c[i, j]$, e la freccia appropriata come valore di $b[i, j]$. Il valore 4 in $c[7, 6]$, quello che si trova nell'angolo inferiore destro della tabella, è la lunghezza della LCS $\langle B, C, B, A \rangle$ di X e Y . Per $i, j > 0$, il valore $c[i, j]$ dipende soltanto dal fatto che $x_i = y_j$ o no e dai valori $c[i-1, j]$, $c[i, j-1]$ e $c[i-1, j-1]$, che sono stati calcolati prima di $c[i, j]$. Per ricostruire gli elementi di una LCS, basta seguire le frecce $b[i, j]$ partendo dall'angolo inferiore destro della tabella; il cammino è indicato dallo sfondo grigio. Ogni freccia " $\nwarrow$ " sul cammino corrisponde a una posizione (evidenziata) per la quale $x_i = y_j$ è un elemento di una LCS.

liario richiesto per calcolare una più lunga sottosequenza comune non diminuisce asintoticamente, perché a ogni modo la tabella c occupa uno spazio $\Theta(mn)$.

Però, possiamo ridurre il fabbisogno asintotico di memoria di LCS-LENGTH, perché questa procedura usa soltanto due righe alla volta della tabella c : la riga da calcolare e quella precedente (in effetti, come chiede di mostrare l'Esercizio 14.4-4, possiamo utilizzare uno spazio appena un po' più grande di quello richiesto da una singola riga di c per calcolare la lunghezza di una più lunga sottosequenza comune). Questo miglioramento è utile se abbiamo bisogno di calcolare soltanto la lunghezza di una più lunga sottosequenza comune; ma se dobbiamo ricostruire gli elementi di una più lunga sottosequenza comune, la tabella più piccola non può contenere le informazioni necessarie per ripercorrere i passi eseguiti dall'algoritmo in tempo $O(m+n)$.

Esercizi

14.4-1

Trovate una più lunga sottosequenza comune delle sequenze $\langle 1, 0, 0, 1, 0, 1, 0, 1 \rangle$ e $\langle 0, 1, 0, 1, 1, 0, 1, 1, 0 \rangle$.

14.4-2

Scrivete lo pseudocodice per ricostruire in tempo $O(m+n)$ una più lunga sottosequenza comune a partire dalla tabella completa c e dalle sequenze originali $X = \langle x_1, x_2, \dots, x_m \rangle$ e $Y = \langle y_1, y_2, \dots, y_n \rangle$, senza utilizzare la tabella b .

14.4-3

Realizzate una versione memoizzata della procedura LCS-LENGTH che richieda un tempo $O(mn)$.

14.4-4

Spiegate come calcolare la lunghezza di una più lunga sottosequenza comune utilizzando soltanto $2 \cdot \min\{m, n\}$ elementi nella tabella e più uno spazio aggiuntivo $O(1)$. Risolvete lo stesso problema utilizzando $\min\{m, n\}$ posizioni più uno spazio aggiuntivo $O(1)$.

14.4-5

Realizzate un algoritmo che richieda un tempo $O(n^2)$ per trovare la più lunga sottosequenza monotonicamente crescente di una sequenza di n numeri.

★ 14.4-6

Realizzate un algoritmo che richieda un tempo $O(n \lg n)$ per trovare la più lunga sottosequenza di una sequenza di n numeri che sia monotonicamente crescente (suggerimento: notate che l'ultimo elemento di una sottosequenza candidata a essere la soluzione di lunghezza i è grande almeno quanto l'ultimo elemento di una sottosequenza candidata a essere la soluzione di lunghezza $i-1$. Memorizzate le sottosequenze candidate a essere soluzione aggiungendo dei puntatori alla sequenza di input).

14.5 Alberi binari di ricerca ottimali

Supponiamo di voler progettare un programma di traduzione di un testo dall'inglese in lituano. Per ogni occorrenza di una parola inglese nel testo, dobbiamo cercare l'equivalente parola lituana. Possiamo effettuare queste ricerche costruendo un albero binario di ricerca con n parole inglesi come chiavi e le corrispondenti parole lituane come dati satellite. Poiché dovremo cercare ogni parola del testo nell'albero, vogliamo ridurre al minimo il tempo totale impiegato nelle ricerche. Possiamo garantire un tempo $O(\lg n)$ per ogni ricerca, utilizzando un albero rosso-nero o qualsiasi altro albero binario di ricerca bilanciato. Le parole, però, si presentano con frequenze differenti e potrebbe accadere che una parola frequentemente utilizzata, come "the", si trovi lontana dalla radice, mentre una parola raramente utilizzata, come "naumachia", si trovi vicino alla radice. Tale organizzazione rallenterebbe la traduzione del testo, in quanto il numero di nodi visitati durante la ricerca di una chiave in un albero binario di ricerca è pari a 1 più la profondità del nodo che contiene la chiave. Vogliamo che le parole che appaiono nel testo con maggiore frequenza occupino le posizioni più vicine alla radice.¹⁰ Inoltre, potrebbero esserci parole nel testo inglese per le quali non esistono le corrispondenti parole lituane:¹¹ queste parole potrebbero non apparire affatto nell'albero binario di ricerca. Come possiamo organizzare un albero binario di ricerca per minimizzare il numero di nodi visitati in tutte le ricerche, se sappiamo già quanto spesso si presenta ogni singola parola?

¹⁰ Se il testo da tradurre riguarda l'antica Roma, potrebbe essere comodo avere la parola "naumachia" vicino alla radice.

¹¹ Sì, c'è la traduzione in lituano di *naumachia*: è *nomacija*.